2011 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems

Online Missing/Repeated Gate Faults Detection in Reversible Circuits

Masoud Zamani
Northeastern University
Boston, MA, USA
Email: mzamani@ece.neu.edu

Mehdi B. Tahoori
Karlsruhe Institute of Technology
Karlsruhe, Germany
Email: mehdi.tahoori@kit.edu

Abstract

Reversible logic is a computation methodology with the promise of possibly zero-energy consumption by elimination of power dissipation due to information loss. Furthermore, reversible logic has direct application in quantum computing. However, since fault models in reversible circuits are fundamentally different from those used for CMOS VLSI technologies, therefore new testing approaches must be developed to test reversible circuits. It has been shown that two new types of faults in the form of gate missing and gate repeating are more likely to occur in reversible circuits, rather than traditional stuck-at faults.

In this paper, we present an approach for online detection of these types of faults in reversible circuits. In this approach, we modify reversible gates in such a way that they can produce information on the number of cascaded gates. By using such information we add an appropriate reversible gate to detect missing and repeated gate faults. Simulation results on a set of benchmark circuits confirm that the proposed approach can detect 100% of single and more than 97% of multiple missing/repeated gate faults, in average. Since, the modified gates provide a combination of basic logic operation (e.g. NAND and NOR) in a reversible gate, therefore, using these gates effectively reduces area overhead and the number of garbage outputs, compared to previous work.

Keywords

Reversible Logic, Online Testing, Missing/Repeated Gate Fault;

I. Introduction

Reversible logic is considered as a promising design paradigm for ultra-low power computation in future emerging technologies, because of the possibility of nearly energy-free reversible computation [1]. In reversible circuits the value of inputs can be determined by the output values, and vice versa. Therefore, in these circuits the information is not lost and as a result, power dissipation due to information loss is eliminated. However, it should be noted that implementing reversible logic in current VLSI technologies does not lead to reduction in power dissipation since currently the major source of power dissipation is not due to information loss. Although, at current technology, energy losses due to irreversibility are negligible with respect to the overall power dissipation, this may change for emerging non-CMOS technologies. Furthermore, quantum logic operations are based on elementary unitary matrix operations which are inherently reversible [1], [2].

In a reversible circuit, there is a one to one relation between permutations of input vectors and the permutation of output vectors [3]. Therefore, the equality of the number of inputs and outputs is a necessary condition for a reversible gate and circuit [3]. Furthermore, a reversible circuit does not contain any fan-out branch or feedback loop.

Off-line testing of reversible circuits has been well studied. Reversibility leads to full controllability and full observability in these circuits, which simplifies test generation [4]. Therefore, for any reversible circuit, there is a complete test set which detects all single stuck-at faults as well as all multiple stuck-at faults in the circuit [5]. On the other hand, a set of techniques for online error detection have been proposed. These techniques can be categorized as parity-generating (e.g. [6]), parity-preserving (e.g. [7]), and dual rail online error detection methods [8], [9]. In our previous work, we have also proposed reversible majority voters for simultaneously masking and diagnosing of stuck-at faults [10].

All previously proposed online testing methods for reversible circuits target stuck-at fault model. However, the stuck-at fault model is not appropriate to model all types of failures in these circuits [11]. This is due to the fact that the stuck-at fault model assumes a permanent connection between a wire and the voltage source. However, in quantum circuits (as a realization of reversible circuits) there is no physical wire [19].

Therefore, specific fault models including *Missing Gate Fault* (MGF) and *Repeated Gate Fault* (RGF) models for reversible circuits have been proposed. In these models, fault is characterized as elimination (replication) of gates in the circuit [12]. These fault models are based on physical characteristics of reversible circuits inspired from their application in quantum computers. Therefore, reliable design techniques for reversible logic must explicitly address these fault models.

In this paper, we present an online fault detection technique to detect missing/repeated gate faults in reversible circuits. We propose a set of reversible gates to generate the inverted value of one of the inputs on one of the non-functional outputs (garbage outputs). The inverted line in cascaded gates is compared to the expected value to detect any missing or

1550-5774/11 $26.00 © 2011 IEEE
DOI 10.1109/DFT.2011.56

IEEE computer society

repeated gate fault. Simulation results on a set of benchmark circuits show that the proposed technique detects all single MGFs and RGFs, and more than 97% of multiple MGFs and RGFs while it reduces the number of garbage outputs by 28% and 35% in average, compared to Toffoli Gates [13] and Fredkin Gate [14], respectively. It also reduces the number of gates by 20% and 31% in average, compared to Toffoli Gates and Fredkin Gates, respectively.

The organization of the rest of the paper is as follows. Section II briefly introduces preliminaries and related work. The modified gates and proposed technique are presented in Section III. Simulation results are presented in Section IV. Finally, Section V concludes the paper.

II. Preliminaries and Related Work

A. Preliminaries

A reversible function is a function which supports the computation in the system in both backward and forward directions (input to output and output to input directions). This logic is realized by a set of reversible gates. In a reversible gate, the input vector can always be retained from the output vector, and vice versa. Therefore, each input permutation is mapped to a unique output permutation. To have the ability of the one to one mapping between input and output vectors, the number of outputs of the gate must be the same as the number of its inputs [3].

However, these gates have more inputs than conventional gates (and hence, more outputs). Inputs of these gates are divided into two groups: *target* inputs and *control* inputs. Target inputs are those inputs which are used in logical computation. On the other hand, control inputs which are the extra inputs added due to reversibility are used to control the logical operation on the functional outputs. Similarly, the outputs of these gates are divided to *Functional* and *Garbage* outputs. Logical computations are produced on functional outputs. Garbage outputs which are the additional output lines added to preserve the reversibility, usually are not used.

It must be noted that, all traditional logic gates (except inverters) require garbage outputs as well as control inputs to satisfy one to one mapping between inputs and outputs. A reversible circuit consists of only reversible gates without any fan-out or feedback loop in the circuit.

B. Related Work

Reversibility in the circuit significantly simplifies off-line testing compared to irreversible circuits. It has been shown that there is a test set which can detect all single/multiple stuck-at faults in a reversible circuit [4]. Variety of parity generating techniques in reversible circuits have been proposed [15], [16], [17]. The general idea behind these techniques is generating a parity before (reference parity) and after (checking parity) the computation. Each computational stage in these techniques consists of two cascaded blocks. The logical operation of a stage is implemented in the first block which generates functional outputs as well as a parity bit of the functional outputs. The second block also generates another parity for those outputs. Any fault resulting in incorrect parity can be detected by comparing these two parity bits. Parity preserving is another approach for online detection of stuck-at faults in reversible circuits [7]. A class of reversible logic gates has been introduced in which the parity of the outputs matches that of the inputs.

In previous work we have proposed a dual rail technique for single stuck-at fault detection during runtime operation [8]. This work has been extended to detect multiple stuck-at faults [9]. In addition to this, we have proposed a majority voter which is robust against single point of failure [10]. This voter provides diagnosis information while it masks stuck-at faults.

Aforementioned techniques target stuck-at to model the faults. However, it is shown that this model is not well suited for reversible circuits. Fault models for reversible circuits have been reviewed in [19]. New logical fault models based on the concepts of missing and repeated logic gate at logical level have been introduced in [11], [12]. These fault models are more realistic for reversible logic implementations compared to traditional CMOS VLSI fault models, e.g. single stuck-at fault [20]. These models are called *Single Missing Gate Fault* (SMGF) model, *Multiple Missing Gate Fault* (MMGF), *Partial Missing Gate Fault* (PMGF), and *Repeated Gate Fault* (RGF). SMGF is a fault model which assumes a reversible gate is removed (completely) from the circuit. MMGF assumes more than a gate (but all are consecutive to each other) are removed (completely) from the circuit. RGF assumes a gate is replaced by two consecutive copies of that gate. PMGF assumes a subset of control inputs of a gate is removed [11], [12]. In other words, if a gate has k control inputs, removing m of these k control inputs results in PMGF (called *m-order* PMGF) [19].

III. Proposed Online Missing/Repeating Gate Detection Approach

A. Missing/Repeated Gate Fault

First, we propose a method to detect SMGF and RGF, and then it is shown how other reversible faults can be addressed by the proposed method. When a gate is missing, its inputs are directly connected to its outputs. In other words, we can consider the missing as buffering the inputs on the outputs. On the other hand, a repeated gate fault leads to replacing a gate with two consecutive gates of the same type, but the order of connection is same for both gates [19]. Figure 1.a shows a reversible circuit consisting of two gates. Missing and repeated gate faults on gate R_2 of this figure are shown in Figure 1.b and Figure 1.c, respectively.

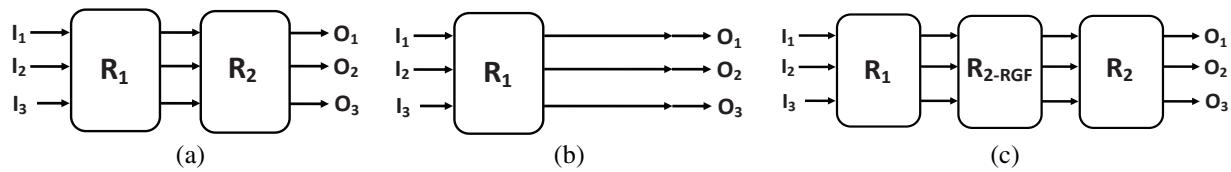


Figure 1. An example of fault effect on a reversible circuit: a) A fault-free circuit, b) SMGF on gate R_2, and c) RGF on gate R_2

B. Proposed Gates

The objective of the previously proposed reversible gates was preserving reversibility in logic computation [13], [14]. However, they are not efficient for online detection of missing/repeated gate faults.

The basic idea behind our proposed gates is keeping information of the number of gates in order to detect missing/repeated gate faults. This can be done by creating inversion of one of the control inputs on one of the garbage outputs. This garbage output can be used as a control input to the successor gate. In the cascaded gates, again the inversion on the same signal is done. In other words, this approach holds the parity information based on the level (stage) of each gate. Therefore, if one of the gates is missing or a gate is repeated, by checking the garbage output at primary outputs the fault can be detected, since the parity information will be distorted. Figure 2 shows the general structure of this method. Figure 2.a shows the fault-free case where the expected value of the first garbage output at the primary output is D. Missing and repeated gate faults are shown in Figure 2.b and Figure 2.c, respectively. In both of these two cases, the expected value on the first garbage output at the primary output is $\neg D$. A simple checker can be used at primary outputs to check D and raise appropriate error signal.

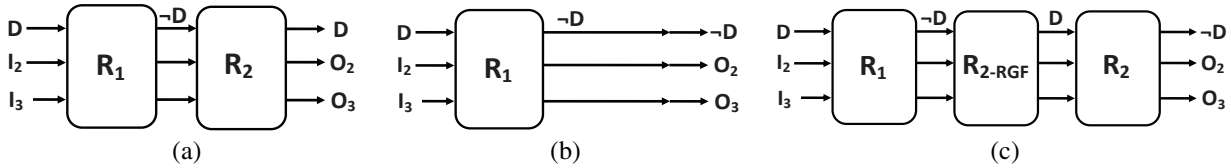
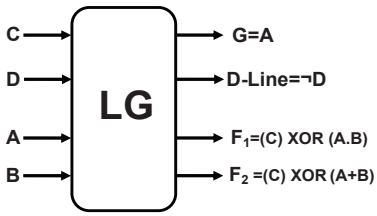


Figure 2. General scheme of online missing/repeated gate detection: a) A fault-free circuit which produces D on the first primary output, b) Gate missing fault on gate R_2 which results in $\neg D$ on the first primary output, and c) Repeated gate fault on gate R_2 which results in $\neg D$ on the first primary output

The proposed gates must be universal gates to be used in any (reversible) logic implementation. Furthermore, since fan-out branches are not allowed in reversible circuits, these gates must be able to provide multiple copies of a signal for reversible implementation of fan-out branches. In the follows, the proposed reversible gates will be introduced.

1) Proposed Reversible Logic Gate (LG): The schematic of the proposed reversible *Logic Gate* (LG) is shown in Figure 3.a. Inputs A and B are the target inputs (computation is done on these inputs), C is the control input (to control gate operation), and D is the detection line (to propagate information of the number of gates). Outputs F_1 and F_2 are the function outputs, G is the garbage output, and $\neg D$ is the propagation line for the detection input. The truth table of this gate is presented in Figure 3.b.

In this gate, if $C = 0$, $A \cdot B$ (logical AND) is computed on F_1 while $A + B$ (logical OR) is computed on F_2. On the other hand, if $C = 1$, logical NAND and NOR are computed on F_1 and F_2, respectively. The garbage output produces A



(a)

	Odd Stage								Even Stage							
Target Inputs	C D = 0 0				C D = 1 0				C D = 0 1				C D = 1 1			
A B	G	¬D	F_1	F_2	G	¬D	F_1	F_2	G	¬D	F_1	F_2	G	¬D	F_1	F_2
0 0	0	1	0	0	0	1	1	1	0	0	0	0	0	0	1	1
0 1	0	1	0	1	0	1	1	0	0	0	0	1	0	0	1	0
1 0	1	1	0	1	1	1	1	0	1	0	0	1	1	0	1	0
1 1	1	1	1	1	1	1	0	0	1	0	1	1	1	0	0	0
			A.B	A+B			¬(A.B)	¬(A+B)			A.B	A+B			¬(A.B)	¬(A+B)

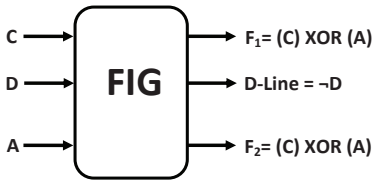
(b)

Figure 3. The proposed reversible *Logic Gate* (LG): a) The schematic of the gate b) The truth table of the gate

to be used in the next stage (if needed). However, the detection line is the line which is used to keep the information of the number of the gates in a path. It produces the inverted value of D regardless of the value of the control input.

Gates are levelized based on their distance from primary inputs. All primary inputs are in level 0. The stage of each gate is one plus the level of the input which is the maximum level among all inputs of that gate. All D-lines at the primary inputs (D inputs at level 0) are constant 0. Therefore, D inputs of all gates at stage 2 are 1. Similarly, all gates at odd stages are fed by $D = 0$ and all gates at even stages are fed by $D = 1$. However, the gates are designed in such a way that their operation is independent of their stage. In other words, for both odd stages and even stages (as seen in the truth table), the gate has the same behavior for logic computation.

2) Proposed Fan-Out/Inversion Gate (FIG): Since, fan-out branches on wires are not allowed in reversible circuits, fan-out must be implemented through a reversible gate. This gate must copy the target input in two functional outputs. Figure 4 shows the schematic and truth table of FIG. In this gate, if the control input (C) is 0, then the input A is copied in F_1 and F_2. Otherwise, $\neg A$ is copied on these two functional outputs. The operation on D-line for FIG is the same as its operation in LG.



(a)

	Odd Stage						Even Stage					
	C D = 0 0			C D = 1 0			C D = 0 1			C D = 1 1		
A	¬D	F_1	F_2	¬D	F_1	F_2	¬D	F_1	F_2	¬D	F_1	F_2
0	1	0	0	1	1	1	1	0	0	1	1	1
1	1	1	1	1	0	0	1	1	1	1	0	0
		A			¬A			A			¬A	

(b)

Figure 4. The proposed reversible *Fan-out/Inversion Gate* (FIG): a) The schematic of the gate b) The truth table of the gate

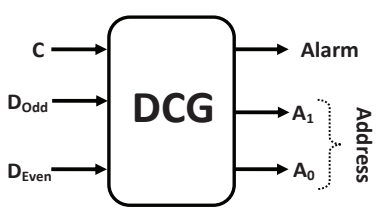
C. Logic Conversion

The proposed gates are used in converting conventional irreversible circuits to reversible one. The general steps to convert a combinational circuit to a reversible circuit consisting of the proposed gates (LG and FIG) are as follows.

1) *Library Cell Definition*: The proposed gates are able to produce a subset of basic logic operations (i.e. AND, NAND, OR, and NOR). Therefore, in the first step, a library must be defined to be used during the logic synthesis. This library consists of the basic logic operations which can be computed by the proposed gates. The library which is used in synthesizing to our proposed gates consists of 2 input gates of AND, NAND, OR, and NOR operations.
2) *Logic Synthesis*: Using the defined library cell, a combinational circuit can be synthesized to a set of gates. Any logic synthesis tool can be used to convert a combinational logic function to a circuit consisting of the proposed gates. However, we have used the SIS synthesis tool [21] for the logic synthesis. It must be noted that, the output of the synthesis tool is still an irreversible circuit. It is due to the fact that, it contains fan-out branches on some of the signals.

3) *Gate Merging*: The proposed gates are able to produce two logic functions in a gate (e.g. F_1 and F_2 can produce NAND and NOR, respectively, on the same reversible gate). This capability can be used to reduce the number of gates in the circuits. Two gates can be merged if:
   a) Both of the gates are fed by the same inputs.
   b) One of the gates is an AND gate and the other one is an OR gate, or one of the gates is a NAND gate and another one is a NOR gate.
4) *Control Input Fixing*: Depending on the logic function, the control inputs for each gate are determined. If a gate must produce AND/OR operation, then the control input of that gate must be fixed to 0. For the gates with NAND and NOR operations the control input must be fixed to 1.
5) *Levelizing*: The gates are levelized based on the level of their inputs. All primary inputs are at level 0. The level of all other signals is equal to the level of the gate which produces that signal. The level of a gate is one plus the level of its inputs (maximum level of inputs).
6) *Fan-out Reduction*: The proposed gates can produce one of the inputs on the garbage output. This property can be used to reduce the number of fan-out branches. If a signal is used as an input to a gate at level i and another gate at level j ($i < j$), then the garbage output of the gate at stage i is used instead of the fan-out branch for the gate at level j.
7) *Fan-out Conversion*: Fan-out branches are not allowed in reversible circuit. Therefore, after the "Fan-out Reduction" step, any fan-out branch must be implemented using FIGs.
8) *Re-levelizing*: After FIG insertion, the level of the gates may change. Therefore, their levels are updated after the "Fan-out Conversion" step.
9) *D-line Connection*: The connections of all signals (except D-line) are determined based on the connections after logic synthesis and conversion steps. However, the connections of D-lines are independent of the logic mapping. The connections of D-lines are done based on the levels of the gates. After the re-levelizing step, for each gate at level s we assign a number which determines the D-path which goes through that gate. To determine the D-path number, for all gates at level s we assign a unique number i ($1 \leq i \leq k$). In this assignment, k is the number of gates at level s. Afterwards, the D-line of all of the gates which have the same assigned number (i) are connected together (these gates are in different levels). Therefore, after this step, there are k D-paths (k is equal to the maximum number of gates at each level).

D. Detection Scheme



(a)

			C = 1			C = 0		
	D_{Odd}	D_{Even}	Alarm	A_1	A_0	Alarm	A_1	A_0
Both Fault-free	1	0	0	0	0	0	1	1
Odd faulty	0	0	1	1	0	0	1	0
Even faulty	1	1	1	0	1	0	0	1
Both faulty	0	1	1	1	1	1	0	0

(b)

Figure 5. *D-Collector Gate* (DCG): a) The schematic of the gate b) The truth table of the gate

A multi-stage circuit consists of a set of gates in each stage. The number of gates in each stage is defined as the depth of that stage. The maximum depth determines the total number of D-paths. Therefore, in a circuit there are k paths which contain D-lines (k: the number of D-paths). Some of these paths contain an odd number of gates and the rest contains an even number of gates. All D inputs of all gates at stage 1 are connected to 0. Therefore, the paths with an odd number of gates result in D at the end which must be equal to 1. On the other hand, the primary outputs of D-paths with an even number of gates are 0. If a gate or an odd number of gates in a D-path is missed, then the expected output of that D-path will be changed. This change can be determined by an "Alarm" signal produced by a gate called *D-Collector Gate* (DCG). This gate is designed based on the fact that all D-inputs at level-0 are 0. It collects a D at an odd stage (call it D_{Odd}) and a D at an even stage (call it D_{Even}). DCG is shown in Figure 5. This gate combines a pair of D-paths, one from an odd path and another from an even path. After combining all pairs of D-paths, for the rest of D-paths (if the number of even

and odd paths are not equal), the corresponding inputs can be replaced by constant 0 and 1, respectively. For example, if there are 4 D-paths containing an odd number of gates, and 5 D-paths containing an even number of gates. Then 4 pairs of paths (each pair contains one odd D-path and one even D-path) are combined by four DCGs. The remaining even D-path is combined by another DCG whose D_{odd} input is connected to 1. Each DCG produces two other outputs (in addition to Alarm signal). These outputs, called A_1 and A_0, indicate the address of the faulty path for diagnosis purposes.

Figure 6 shows the detection scheme on two paths. For the sake of simplicity, the rest of the inputs of the gates are not shown in the figure. In this example, one of the D-paths (indicated by red dotted line) consists of 3 gates, while another D-path (indicated by green dashed line) consists of 4 gates. These two paths for D-lines are collected by a DCG. If one or both paths contain an odd number of faults, it is detected by the Alarm signal.

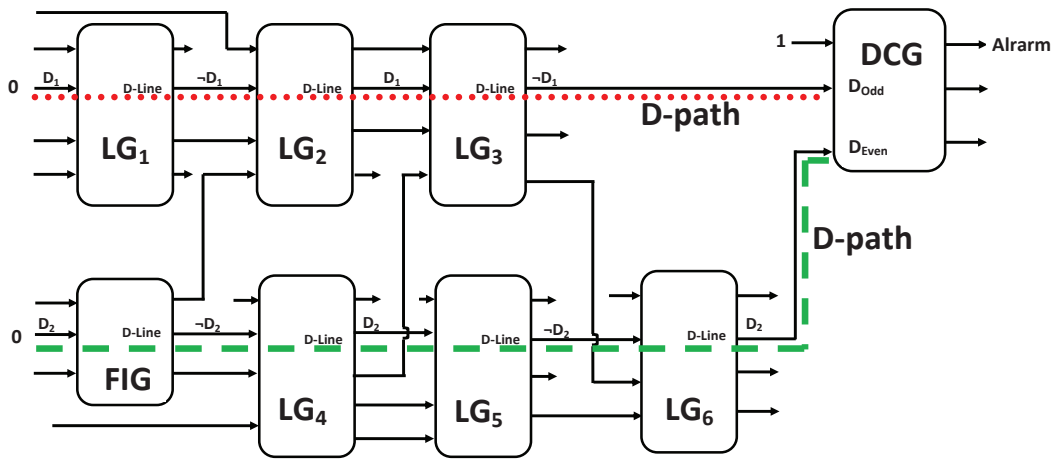


Figure 6. Detection scheme for a circuit consists of two D-paths

E. Discussion

The proposed gates are designed to detect SMGF and RGF. However, they are able to detect MMFG as long as the number of missing gates is odd. On the other hand, this approach can detect multiple faults as long as the error is not masked in at least one of the paths collected by a DCG. The error is masked on a path i if $|N_m^i - N_r^i|$ is even on this path, where N_m^i and N_r^i are the number of missing and repeating gate faults on path i, respectively.

PMGF is defined as missing a subset of control inputs of a K-CNOT gate [11]. If the same concept is applied to the proposed gates, and we assume D is a control input (it is not a target input), then missing of a control input can be translated as short connection between that control input and the corresponding output. Missing of D means instead of $\neg D$, the gate produces D, which is detected. Both LG and FIG have two control inputs (C and D). For these gates, there are two types of PMGFs, namely first-order PMGF and second-order PMGF. It is clear that half of the first-order PMGFs in these gates leads to missing of D line. Therefore, 50% of first-order PMGFs are detected. On the other hand, all second-order PMGFs on these gates leads to missing of D-line, which are also detected. DCG has a control input which is 1 in the fault-free circuit. If due to PMGF in this gate, C is directly connected to the Alarm output, then the Alarm will be 1 which is an indication of fault by default. 50% of first-order PMGF as well as 100% of second-order PMGF in LG and FIG are detected. Therefore, 75% of PMGFs are detected in these gates, in average. On the other hand, 100% of PMGFs in DCG are detected.

IV. Simulation Result

The proposed gates have been compared to previous work (Toffoli Gate [13] and Fredkin Gate [14]) in terms of area (number of gates and input nets), delay (maximum gate level), and number of garbage outputs (number of unused outputs). To do so, a set of MCNC benchmark circuits has been mapped based on the logic conversion method introduced in the previous section. Table II and Table I compare our proposed method to Toffoli Gate and Fredkin Gate in mapping of different benchmark circuits.

In Table I, the maximum number of gates in all the paths is selected for delay comparison. In other words, a path from the primary inputs to the primary outputs which contains the maximum number of gates is used in delay estimation. In this table, the number of garbage outputs is calculated by counting the number of unused outputs.

Table I
EXPERIMENTAL RESULTS ON A SET OF BENCHMARKS; THE NUMBER OF GARBAGE OUTPUTS AND DELAY

	Toffoli Gate [13]		Fredkin Gate [14]		Proposed Gates	
Circuit	Garbage	Delay	Garbage	Delay	Garbage	Delay
alu2	2560	19	2856	19	1859	18
b9	1001	14	1224	16	815	13
majority	25	6	36	8	25	6
t481	14135	22	15109	24	9712	22
term1	5628	19	6162	21	3835	17
vda	3536	17	3814	19	2653	17
x4	7010	18	8117	20	5203	16
Average	4842	16.4	5331	18.1	3446	15.6

Toffoli and Fredkin gates consists of 3×3 reversible gates. However, our proposed gates are 4×4 logic gate (LG) and 3×3 fan-out (FIG) and collector (DCG) gates. Therefore, area estimation of the circuits depends on the physical realization of the gates. However, in Table II, the relative area of the circuits are reported. In order to have a relative area estimation, the total number of the gates as well as the total number of the input nets for each reversible mapping method are reported in the table. Since LG has the capability of producing two basic logic operation (e.g. AND and NAND) on a gate, therefore the total number of gates mapped on proposed gates is less than the other methods. Reversible circuits are proposed to be implemented on a technology in which dominant power consumption source is due to information loss. In such a technology these circuits consume near-zero energy. Therefore, it is meaningless to report on power consumption of reversible circuits mapped into CMOS technology in which the dominant source of power is not due to information loss.

Table II
EXPERIMENTAL RESULTS ON A SET OF BENCHMARKS; THE NUMBER OF GATES AND THE NUMBER OF INPUT NETS

	Toffoli Gate [13]		Fredkin Gate [14]		Proposed Gates				
Circuit	Gates	Inputs	Gates	Inputs	Total Number of Gates	LG	FIG	DCG	Inputs
alu2	1715	5145	3895	11685	1385	746	533	106	4901
b9	655	1965	764	2292	587	367	172	48	2128
majority	17	51	22	66	17	14	1	2	65
t481	9436	28308	9918	29754	7283	3878	2878	529	25733
term1	3761	11283	4021	12063	2882	1605	1049	228	10251
vda	2398	7194	2521	7563	1948	1149	675	124	6993
x4	4688	14064	5238	15714	3887	2150	1409	328	13811
Average	3238	9714	3768	11304	2569	1415	959	195	9122

Table III shows the coverage of the proposed gates for different fault rates for multiple fault detection analysis. For each fault rate, we have randomly injected faults to the circuit. The number of inserted faults is determined by the fault rate. For example, if there are 100 gates in the circuit and the fault rate is 10%, then we inject 10 faults to the circuit. As mentioned in the previous section, multiple faults are detected as long as the error is not masked in at least one D-path. It must be noted, since the probability of all paths containing even number of faults is very small, therefore the proposed method can detect multiple faults in most of the cases. The fault simulation of each circuit is repeated $100,000$ times for each fault rate. The average number of detection (at least one D-path with odd number of faults) for these $100,000$ simulations has been reported in Table III. The second column of this table (# of D-paths) shows the number of D-paths in each circuit.

Table III
ONLINE DETECTION COVERAGE ACHIEVED BY THE PROPOSED METHOD FOR DIFFERENT FAULT RATES

		Fault rate					
Circuit	# of D-paths	5%	10%	15%	20%	25%	Average
alu2	213	99.97%	99.97%	99.97%	99.97%	99.97%	99.97%
b9	96	99.78%	99.80%	99.80%	99.80%	99.79%	99.79%
majority	5	100.00%	96.92%	96.72%	96.79%	96.84%	97.39%
t481	1057	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
term1	455	100.00%	99.91%	99.97%	100.00%	100.00%	99.97%
vda	248	99.93%	99.98%	100.00%	100.00%	100.00%	99.99%
x4	656	100.00%	99.96%	99.89%	99.68%	99.91%	99.88%

V. Conclusion

Reversible logic is a novel computing scheme with the promising feature of possibly zero-energy consumption by elimination of power dissipation due to information loss. In this paper, we presented a new approach to detect repeated and missing gate faults. In this approach, we modify reversible gates in such a way that they can produce parity information about the level of the gate in the circuit. Using such information we design a checker to detect missing or repeated gate faults. Simulation results on a set of benchmark circuits confirm that the proposed approach can detect 100% single and more than 97% of multiple missing/repeated gate faults as well as majority of partial missing gate faults, in average. In addition, the proposed method reduces the number of garbage outputs by 28% and 35% in average, compared to Toffoli Gates and Fredkin Gate, respectively. It also reduces the number of gates by 20% and 31% in average, compared to Toffoli Gates and Fredkin Gate, respectively.

References

[1] J. Donald and N. K. Jha. Reversible logic synthesis with Fredkin and Peres gates. *J. Emerg. Technol. Comput. Syst.*, 4(1):1–19, 2008.

[2] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. Smolin, and H. Weinfurter. Elementary gates for quantum computation. *J. Phy. Rev. A*, 52:3457–3467, 1995.

[3] H. M. H. Babu and A. R. Chowdhury. Design of a reversible binary coded decimal adder by using reversible 4-bit parallel adder. *VLSI Design'05*, 255–260, 2005.

[4] K. N. Patel, J. P. Hayes, and I. L. Markov. Fault testing for reversible circuits. *TCAD*, 23(8):1220–1230, 2004.

[5] K. N. Patel, J. P. Hayes, and I. L. Markov. Fault testing for reversible circuits. *VTS'03*, 410–416, 2003.

[6] D. P. Vasudevan, P. K. Lala, and J. P. Parkerson. A novel approach for on-line testable reversible logic circuit design. *ATS'04*, 325–330, 2004.

[7] B. Parhami. Fault-tolerant reversible circuits. *ACSSC '06*, 1726–1729, 2006.

[8] N. Farazmand, M. Zamani, and M. B. Tahoori. Online fault testing of reversible logic using dual rail coding. *IOLTS'10*, 204–205, 2010.

[9] N. Farazmand, M. Zamani, and M. B. Tahoori. Online Multiple Fault Detection in Reversible Circuits. *DFT'10*, 429–437, 2010.

[10] M. Zamani, N. Farazmand, and M. B. Tahoori. Fault masking and diagnosis in reversible circuits. *ETS'11*, 69–74, 2011.

[11] I. Polian, T. Fiehn, B. Becker, and J. P. Hayes. A family of logical fault models for reversible circuits. *ATS'05*, 422–427, 2005.

[12] J. P. Hayes, I. Polian, and B. Becker. Testing for missing-gate faults in reversible circuits. *ATS'04*, 100–105, 2004.

[13] T. Toffoli. Reversible computing. *Automata, Languages and Programming*, 632–644, 1980.

[14] E. Fredkin and T. Toffoli. Conservative logic. *Int. J. of Theo. Phy.*, 21: 219–253, 1982.

[15] D. Jia D. P. Vasudevan, P. K. Lala and J. P. Parkerson. Reversible-logic design with online testability. *TIM*, 55(2):406–414, 2006.

[16] S. N. Mahammad and K. Veezhinathan. Constructing online testable circuits using reversible logic. *TIM*, 59(1):101–109, 2010.

[17] H. Thapliyal and A. P. Vinod. Designing efficient online testable reversible adders with new reversible gate. *ISCAS'07*, 1085–1088, 2007.

[18] J. Mathew, J. Singh, A. A. Taleb, and D. K. Pradhan. Fault tolerant reversible finite field arithmetic circuits. *IOLTS '08*, 188–189, 2008.

[19] I. Polian and J.P. Hayes. Advanced modeling of faults in reversible circuits. *EWDTS'10* , 376–381, 2010.

[20] Z. Jing and J. C. Muzio. Analyzing fault models for reversible logic circuits. *CEC'06*, 2422–2427, 2006.

[21] SIS synthesis tool documentation: http://embedded.eecs.berkeley.edu/pubs/downloads/sis/index.htm. 1994.